

コンパイラ (9)

2026/10/23

抽象構文

$$\begin{array}{lcl} & S & \\ E \rightarrow E + T \mid T & = & E \\ T \rightarrow T * F \mid F & & T \\ F \rightarrow ID \mid (E) & & \\ S \rightarrow ID = E & & ID * \end{array}$$

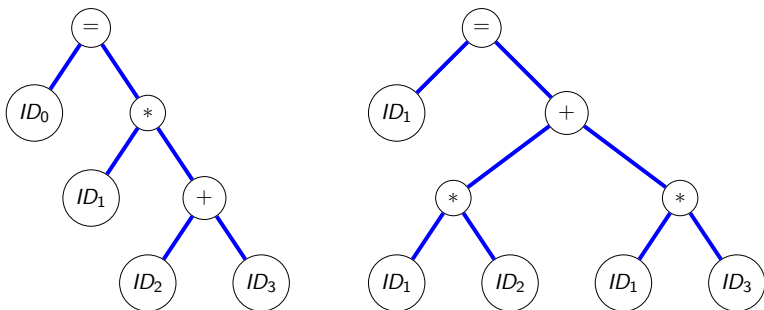
抽象構文木

解析木からコード生成に必要な部分を残して変形したもの

内部接点＝演算子、葉＝オペランド

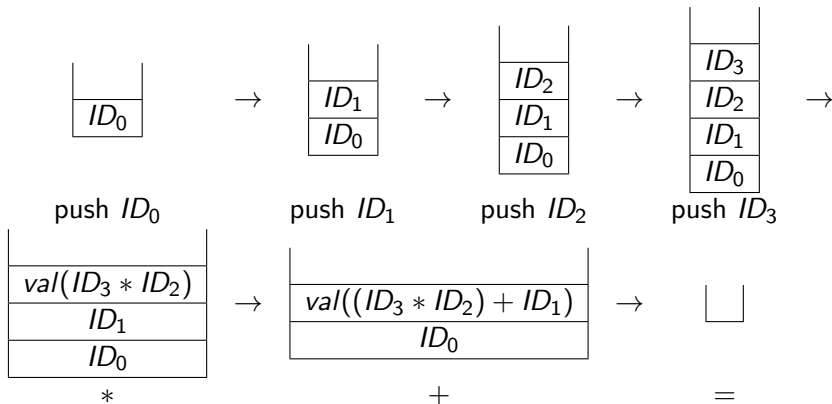
構文主導型変換によって中間コードを生成する際に便利

等価な意味を持つ抽象構文木＝最適化に利用



逆ポーランド記法

木の後置記法: スタックを用いた実行に対応



3 番地コード

$C = A \theta B$ $+, -, *, /, \text{and}, \text{or}, >, >=, =, ! =, <=, <$

$B = \theta A$ $-, \text{not}$

$B = A$

Jump on condition

if A goto L

if $\text{not} A$ goto L

Unconditional Jump

goto L

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

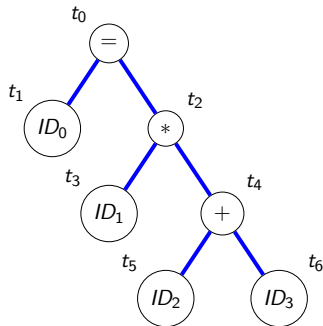
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



抽象構文木に一時変数 t_i を割り当てる

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

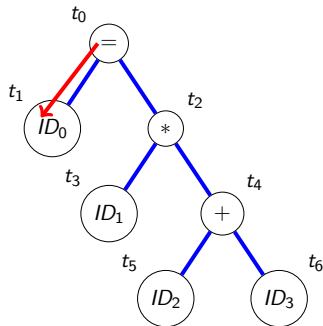
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし, $t_1 = ID_0$ を生成

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

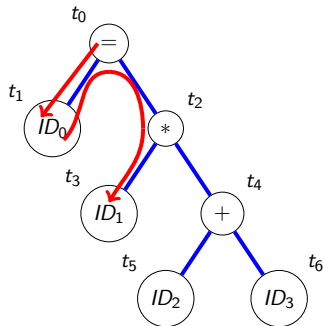
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし、 $t_3 = ID_1$ を生成

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

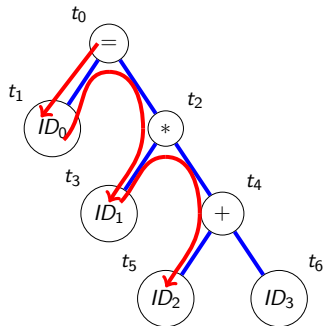
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし， $t_5 = ID_2$ を生成

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

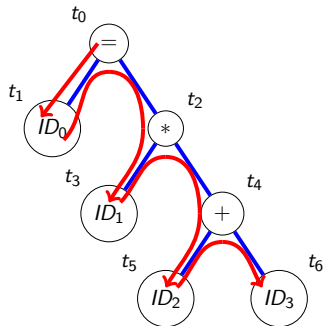
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし、 $t_6 = ID_3$ を生成

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

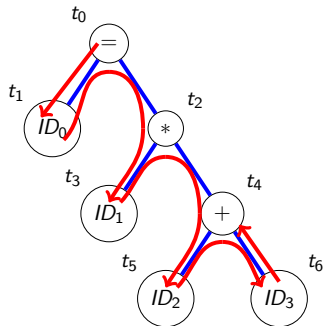
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし、 $t_4 = t_5 + t_6$ を生成

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

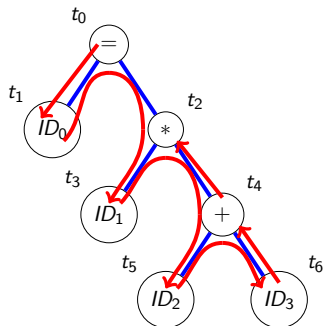
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし， $t_2 = t_3 * t_4$ を生成

抽象構文木からの3番地コード生成

$$ID_0 = ID_1 * (ID_2 + ID_3)$$

$$t_1 = ID_0$$

$$t_3 = ID_1$$

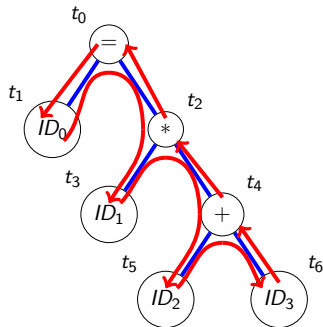
$$t_5 = ID_2$$

$$t_6 = ID_3$$

$$t_4 = t_5 + t_6$$

$$t_2 = t_3 * t_4$$

$$t_7 = (t_1 = t_2)$$



ポストオーダーでトレースし、 $t_0 = (t_1 = t_2)$ を生成

手続き呼び出しの3番地コード

```
int fib(int x) {  
    if (x=0) {  
        return 0;  
    } else if (x=1) {  
        return 1;  
    } else {  
        return fib(x-2)+fib(x-1);  
    }  
}  
  
print fib(4);
```

```
define @fib(x) {  
    t1 := x  
    t2 := (x=0)  
    if not t1 goto l1  
    return 0  
l1: t3 := (x=1)  
    if not t2 goto l2  
    return 1  
l2: t4 := t1 - 2  
    t5 := call fib(t4)  
    t6 := t1 - 1  
    t7 := call fib(t6)  
    t8 := t5 + t7  
    return t8  
}  
  
define @main() {  
    t1 := 4  
    t2 := call fib(t1)  
    print t2  
}
```

定義: `define @関数名 (仮引数リスト)`

呼出: `call @関数名 (実引数リスト)`

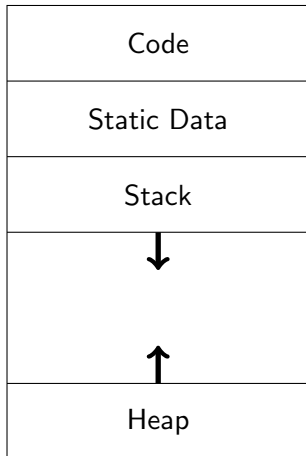
実行時環境

メモリ領域の割り当て方

- 大域変数＝ヒープ領域 (プロセス内のメモリ領域) にわりあてる。

`malloc, free`

- 演習のコンパイラでは、スタック領域の底にわりあてる (pop しない)。
- 局所変数＝スタックにわりあてる
- 関数・手続の引数のわりあて (特殊な局所変数)



静的割当てと動的割当て

静的割当て: 実行の最初から最後までメモリ領域が変化しない
動的割り当て: メモリ領域が実行時に決定
実行終了時に領域が不要になる

```
int a;  
  
int func(int x) {  
    int y;  
    y = x * x;  
    return (y+y);  
}  
  
main() {  
    int b=1;  
    a=2;  
    printf("%d\n",func(a)+b);  
}
```

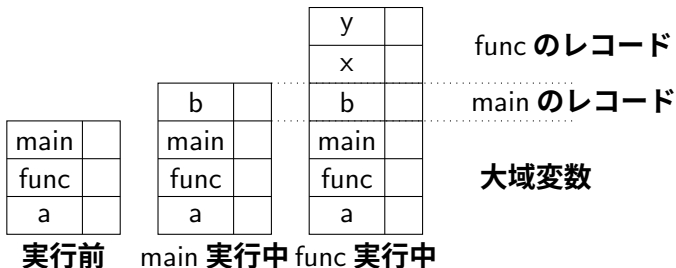
静的割当て: a,func,main

動的割当て: x,y,b

駆動レコード

手続き/関数の一回の実行に必要な局所情報

PC(Program Counter)、返り値、参照可能なシンボルの集合



駆動レコードの構成

一時変数領域
局所データ領域
静的リンク
動的リンク
戻り番地
実引数計算領域
戻り値
退避領域

← FP

関数内での計算の一時領域

局所宣言された変数の領域

次に外へ参照すべきレコード

Return 後に復元すべきレコード

Return 後に再開する PC

呼出の際に引数受け渡しに使う

関数の計算結果を格納

呼出の際の一時領域

変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

?
a=?

変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
→   a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

?
a=2

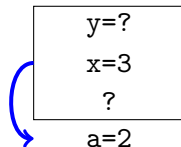
変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
→ printf("func(3)=%d",  
        func(3));  
}
```

?
a=2

変数の参照

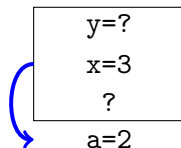
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}  
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

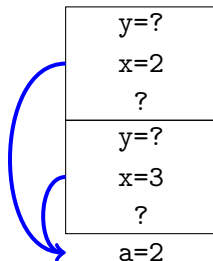
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

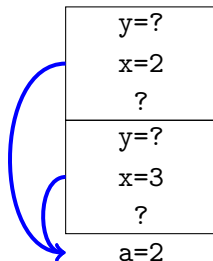
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}  
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

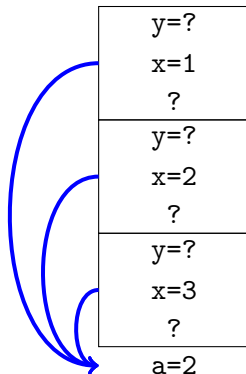
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

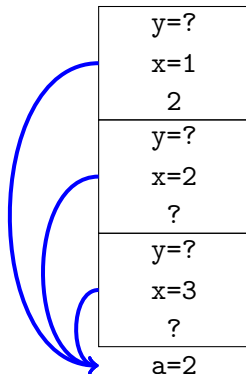
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}  
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

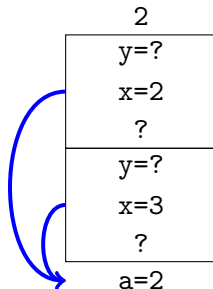
```
int a;  
int func(int x) {  
    int y;  
→  if(x==1) {return(a);}  
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

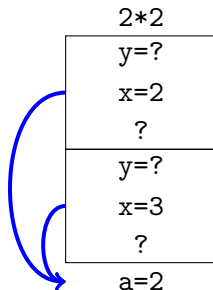
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

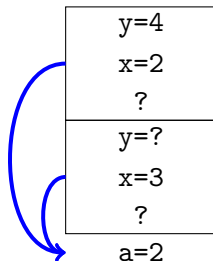
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

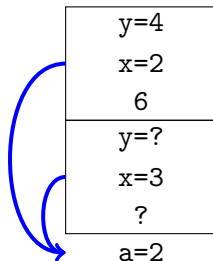
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

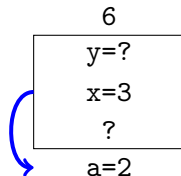
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        → return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

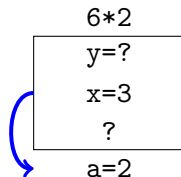
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

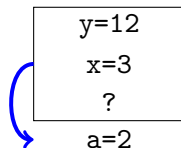
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

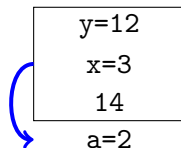
```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
→      y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        →    return(a+y);  
    }  
}  
void main() {  
    a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```



静的リンク

変数の参照

```
int a;  
int func(int x) {  
    int y;  
    if(x==1) {return(a);}   
    else {  
        y=func(x-1)*2;  
        return(a+y);  
    }  
}  
void main() {  
→   a=2;  
    printf("func(3)=%d",  
        func(3));  
}
```

14
a=2

変数の参照/記憶位置の決定

局所変数: フレームポインタからの相対位置

大域変数: ヒープ内のアドレス

記号表にはスタックフレームレジスタからの相対番地＝何番目に宣言でこういった型をもつか。

型情報＝確保する領域量 (実習の場合は整数のみ)

関数の引数うけわたりは局所変数の宣言＋代入

動作意味

$$\langle \{S, E, T, F\}, \{+, *, (,), n\}, P, S \rangle$$
$$\left\{ \begin{array}{ll} S \rightarrow E\#, & T \rightarrow F, \\ E \rightarrow E + T, & F \rightarrow (E), \\ E \rightarrow T, & F \rightarrow n, \\ T \rightarrow T * F \end{array} \right\}$$

各生成規則は値を計算する意味を持つ。

$E \rightarrow E + T$: 右辺の E の値と T の値を加算した値を左辺の E の値とする。

$\Rightarrow$ '+' に加算の意味を与える。

構文に従った動作意味

$$\begin{array}{ll} S \rightarrow E\# & \text{var}(S) = \text{var}(E) \\ E \rightarrow E_1 + T & \text{var}(E) = \text{var}(E_1) + \text{var}(T) \\ E \rightarrow T & \text{var}(E) = \text{var}(T) \\ T \rightarrow T_1 * F & \text{var}(T) = \text{var}(T_1) \times \text{var}(F) \\ T \rightarrow F & \text{var}(T) = \text{var}(T) \\ F \rightarrow (E) & \text{var}(F) = \text{var}(E) \\ F \rightarrow i & \text{temp} = \text{val}(i); \text{var}(F) = \text{temp} \end{array}$$

構文主導型変換

インタプリタ： 生成規則に対応する動作をする。

コンパイラ： 生成規則に対応した目的コード（中間コード）を生成する。

構文主導型変換

CF 文法の各生成規則に対してコード生成ルーチンをわりあてる。

- ▶ `newTemp()`
変数 T の領域を確保して、記号番地または一時変数へのポインタを返す。
- ▶ `newLabel()`
新たなラベル名を返す。
- ▶ `generate(中間コード)`
中間コードを出力ファイルに出力する。

n 回目に `newTemp()` を呼び出したとき、生成される変数 t_n

m 回目に `newLabel()` を呼び出したとき、生成されるラベル L_m

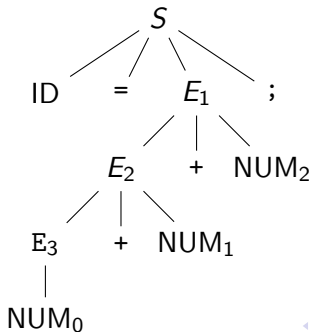
構文主導型変換の基礎

$$S \rightarrow ID = E ;$$

$$E \rightarrow E^{(1)} + \text{NUM}$$

$$E \rightarrow \text{NUM}$$

$$ID = \text{NUM}_0 + \text{NUM}_1 + \text{NUM}_2 ;$$



中間コード (3 番地コード) 生成

生成規則	出力される 3 番地コード
$E_3 \rightarrow \text{NUM}_0$	$t1 = \text{NUM}_0.val$
$E_2 \rightarrow E_3 + \text{NUM}_1$	$t2 = t1 + \text{NUM}_1.val$
$E_1 \rightarrow E_2 + \text{NUM}_2$	$t3 = t2 + \text{NUM}_2.val$
$S \rightarrow \text{ID} = E_1 ;$	$\text{ID.name} = t3$

$E \rightarrow \text{NUM}$	$\{E.var := \text{newTemp}();$ $\quad \text{generate}('E.var = \text{NUM.val}')$ $\}$
$E \rightarrow E^{(1)} + \text{NUM}$	$\{E.var := \text{newTemp}();$ $\quad \text{generate}('E.var = E^{(1)}.var + \text{NUM.val}')$ $\}$
$S \rightarrow \text{ID} = E ;$	$\{\text{generate}('ID.name = E.var')\}$

IF 文の中間コード生成

文 $\rightarrow$ if '(' 式 ')' 文⁽¹⁾

1. '式' のコードを生成 t1 に結果収納
2. if not t1 goto l1
3. '文'⁽¹⁾ のコードを生成
4. l1:

IF 文の中間コード生成

文 $\rightarrow$ if '(' 式 ')' 文⁽¹⁾ else 文⁽²⁾

1. ' 式 ' のコードを生成 t1 に結果収納
2. if not t1 goto l1
3. ' 文'⁽¹⁾ のコードを生成
4. goto l2
5. l1:
6. ' 文'⁽²⁾ のコードを生成
7. l2:

while 文の中間コード生成

文 $\rightarrow$ while '(' 式 ') ' 文⁽¹⁾

1. l1:
2. ' 式 ' のコードを生成 t1 に結果収納
3. if not t1 goto l2
4. ' 文 '⁽¹⁾ のコードを生成
5. goto l1
6. l2:

構文主導型変換によるコード生成

```
文 → if ( { 文.label:=newLabel();}  
式      (式のコード)  
)      {generate('if_not 式.var goto 文.label')}  
文(1)  (文(1) のコード){generate('label 文.label');  
      文(1).label:=newLabel();generate('goto 文(1).label')}  
else    {generate('label 文.label')}  
文(2)    {generate('label 文(1).label')}
```

```
文 → while ( { 文.label1:=newLabel();  
              generate('label 文.label1')}  
式      (式のコード)  
)      { 文.label2:=newLabel();  
          generate('if_not 式.var goto 文.label2')}  
文(1)  (文のコード)  
      {generate('goto 文.label1');  
        generate('label 文.label2')}
```